



**Brit College of Engineering & Technology
(BCET)**

Data Analytics Logbook

Research Project-5

Data Science and AI

(Supervised ML – Classification 1)

Submitted by: Md.Akib Sultan

Student ID:

Email: akibsultan1045@gmail.com

[https://colab.research.google.com/drive/1lumVluc82NxQefd6OapUFBp5iXLaBloi?
usp=sharing](https://colab.research.google.com/drive/1lumVluc82NxQefd6OapUFBp5iXLaBloi?usp=sharing)

Mastering Supervised Machine Learning: A HandsOn Tutorial with the Palmer Penguins Dataset

Supervised Machine Learning is a fundamental branch of AI where algorithms learn from labelled data to make predictions or decisions. Imagine you have a dataset where each data point comes with an answer – like knowing the species of a penguin based on its measurements. Supervised learning algorithms learn this mapping, and then can predict the species of a *new*, unseen penguin.

This comprehensive tutorial will walk you through the entire supervised machine learning pipeline, from data acquisition and cleaning to model training, evaluation, and decision-making. We'll use the delightful **Palmer Penguins dataset**, a popular alternative to the Iris dataset, allowing us to build a classification model.

Dataset: Palmer Penguins

The Palmer Penguins dataset contains measurements for three species of penguins (Adelie, Chinstrap, and Gentoo) collected from islands in the Palmer Archipelago, Antarctica. It includes characteristics like culmen (beak) length and depth, flipper length, body mass, island, and sex.

Our goal will be to **predict the species of a penguin** based on its physical measurements and island. This is a **multiclass classification problem**.

Dataset Source: The dataset is readily available through the seaborn library in Python, which simplifies its loading. For those who prefer a direct CSV, it can also be found on Kaggle: [Palmer Archipelago \(Antarctica\) Penguin Data](#)

The Supervised Machine Learning Workflow

Here's the journey we'll embark on:

1. **Data Loading & Initial Inspection:** Getting acquainted with our data.
2. **Data Cleaning, Preprocessing & Wrangling:** Transforming raw data into a usable format.
3. **Exploratory Data Analysis (EDA):** Uncovering insights and patterns.
4. **Feature Engineering:** Creating new, more informative features (if applicable).
5. **Model Training:** Teaching different algorithms to learn from our data.
6. **Model Evaluation:** Assessing how well our models perform using various metrics

and visualizations.

7. Prediction & Decision Making: Using the model for real-world scenarios.

Google Colab Step-by-Step

Load the Palmer Penguins dataset from seaborn

try:

```
df = sns.load_dataset('penguins')  
print("Dataset loaded successfully from seaborn.")
```

except Exception as e:

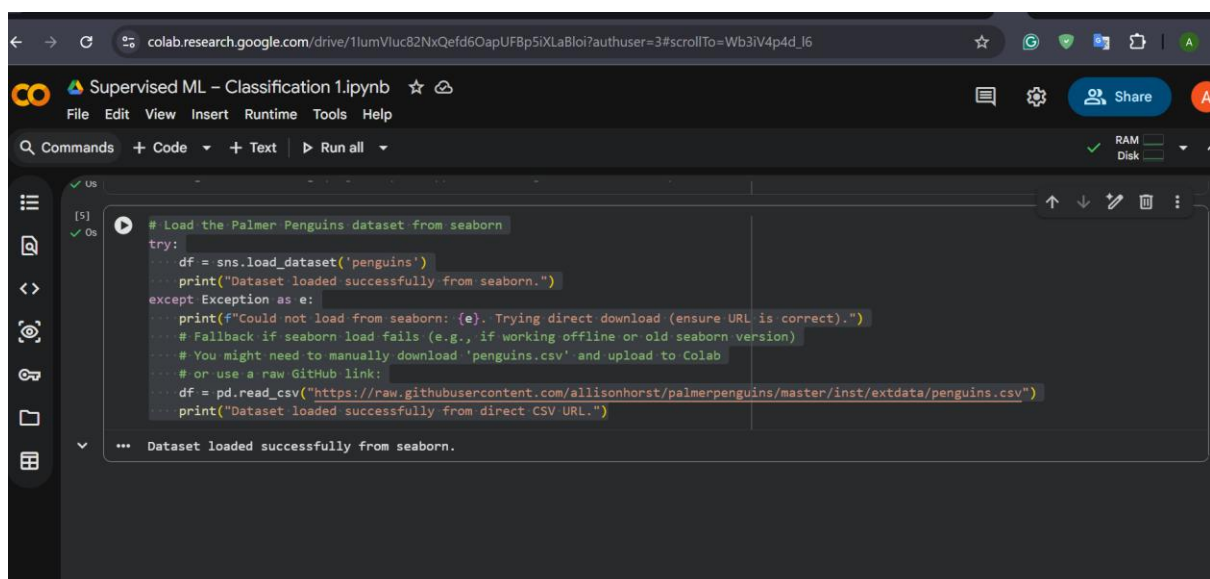
```
print(f"Could not load from seaborn: {e}. Trying direct download (ensure URL is correct).")
```

Fallback if seaborn load fails (e.g., if working offline or old seaborn version)

You might need to manually download 'penguins.csv' and upload to Colab

or use a raw GitHub link:

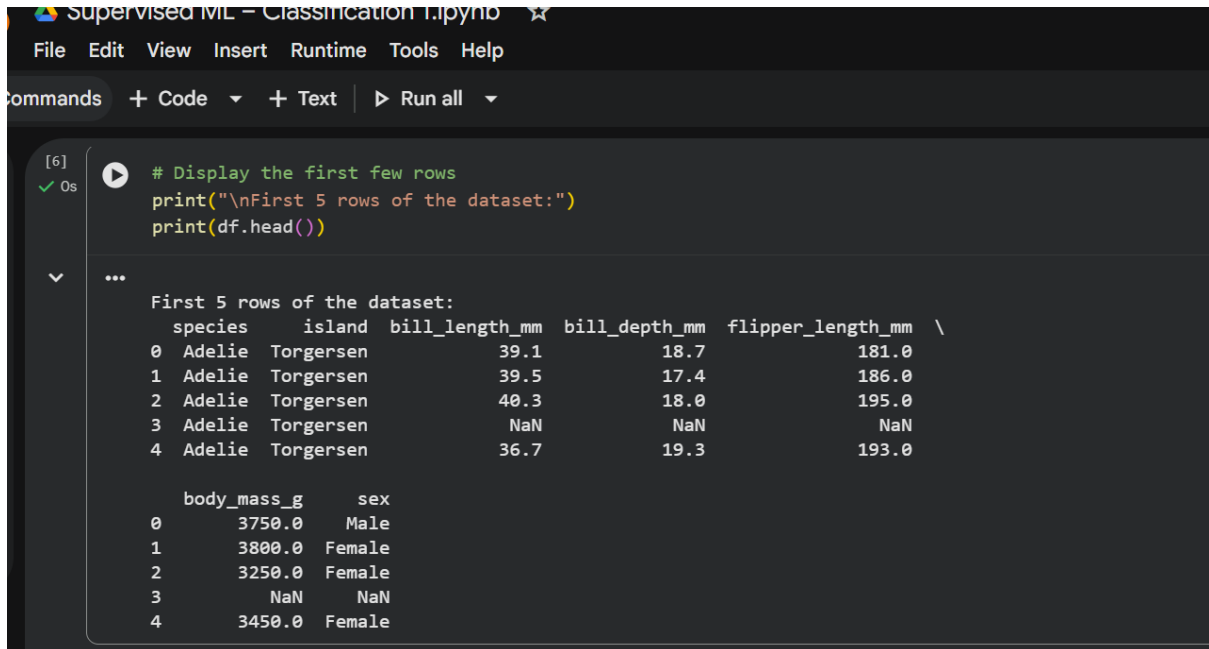
```
df =  
pd.read_csv("https://raw.githubusercontent.com/allisonhorst/palmerpenguins/master/inst/extdata/penguins.csv")  
print("Dataset loaded successfully from direct CSV URL.")
```



```
# Display the first few rows
```

```
print("\nFirst 5 rows of the dataset:")
```

```
print(df.head())
```



Supervised ML – Classification 1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[6] # Display the first few rows
print("\nFirst 5 rows of the dataset:")
print(df.head())
```

First 5 rows of the dataset:

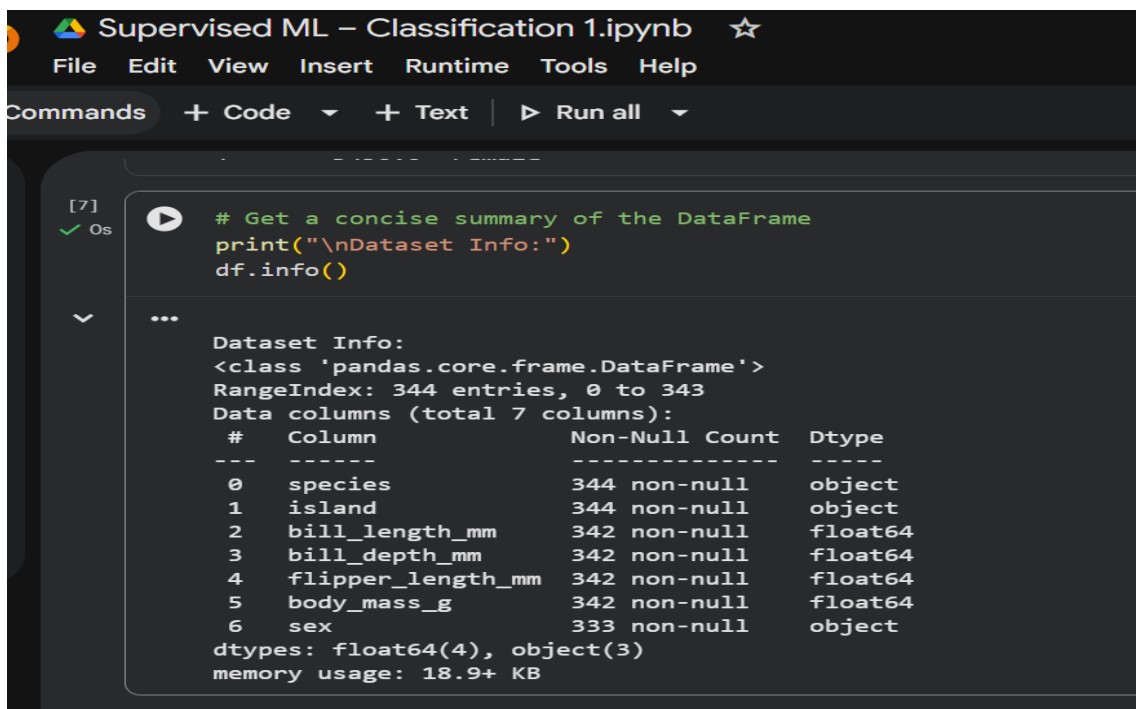
	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	
0	Adelie	Torgersen	39.1	18.7	181.0	
1	Adelie	Torgersen	39.5	17.4	186.0	
2	Adelie	Torgersen	40.3	18.0	195.0	
3	Adelie	Torgersen	NaN	NaN	NaN	
4	Adelie	Torgersen	36.7	19.3	193.0	

	body_mass_g	sex
0	3750.0	Male
1	3800.0	Female
2	3250.0	Female
3	NaN	NaN
4	3450.0	Female

```
# Get a concise summary of the DataFrame
```

```
print("\nDataset Info:")
```

```
df.info()
```



Supervised ML – Classification 1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[7] # Get a concise summary of the DataFrame
print("\nDataset Info:")
df.info()
```

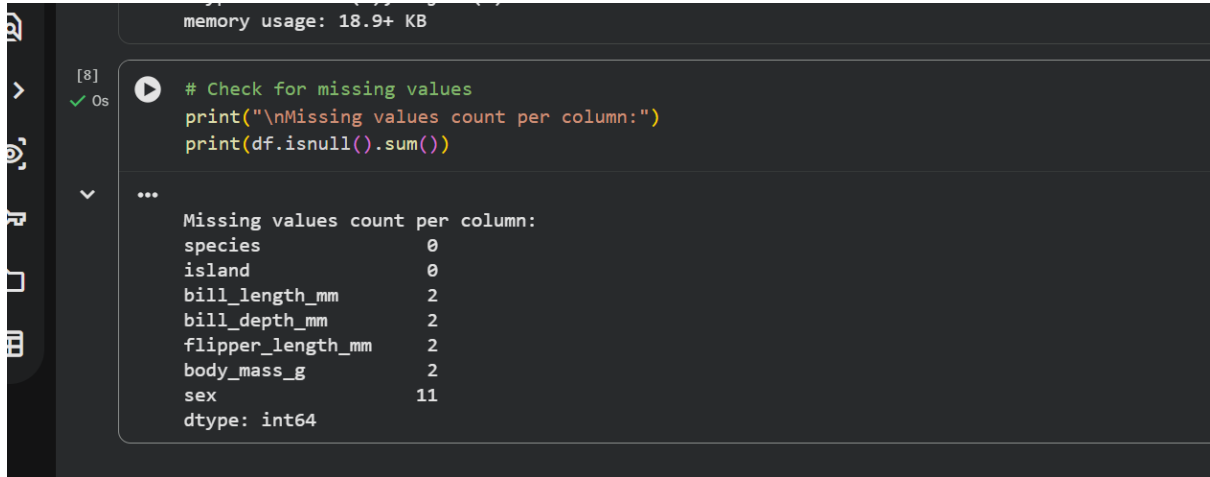
Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  ---                ---
0   species                344 non-null    object
1   island                 344 non-null    object
2   bill_length_mm         342 non-null    float64
3   bill_depth_mm          342 non-null    float64
4   flipper_length_mm      342 non-null    float64
5   body_mass_g            342 non-null    float64
6   sex                    333 non-null    object
dtypes: float64(4), object(3)
memory usage: 18.9+ KB
```

```
# Check for missing values
```

```
print("\nMissing values count per column:")
```

```
print(df.isnull().sum())
```



A screenshot of a Jupyter Notebook cell. The cell number is [8] and it has a green checkmark icon. The code in the cell is:

```
# Check for missing values
print("\nMissing values count per column:")
print(df.isnull().sum())
```

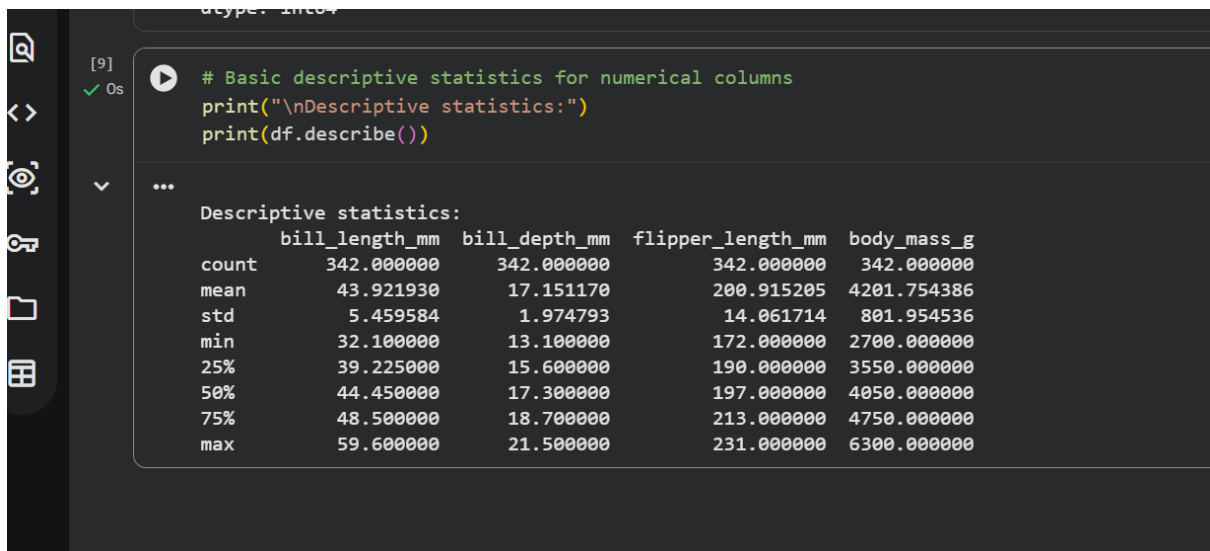
 The output of the cell is:

```
Missing values count per column:
species      0
island       0
bill_length_mm    2
bill_depth_mm    2
flipper_length_mm 2
body_mass_g      2
sex           11
dtype: int64
```

```
# Basic descriptive statistics for numerical columns
```

```
print("\nDescriptive statistics:")
```

```
print(df.describe())
```



A screenshot of a Jupyter Notebook cell. The cell number is [9] and it has a green checkmark icon. The code in the cell is:

```
# Basic descriptive statistics for numerical columns
print("\nDescriptive statistics:")
print(df.describe())
```

 The output of the cell is:

```
Descriptive statistics:
      bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g
count      342.000000      342.000000      342.000000      342.000000
mean         43.921930         17.151170         200.915205      4201.754386
std           5.459584           1.974793           14.061714       801.954536
min           32.100000          13.100000          172.000000      2700.000000
25%           39.225000          15.600000          190.000000      3550.000000
50%           44.450000          17.300000          197.000000      4050.000000
75%           48.500000          18.700000          213.000000      4750.000000
max           59.600000          21.500000          231.000000      6300.000000
```

Observation:

- species, island, sex are categorical.
- bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g are numerical.
- There are missing values

in bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g, and sex.

2. Data Cleaning, Preprocessing & Wrangling

--- Handling Missing Values ---

For simplicity in this tutorial, we'll drop rows with any missing values.

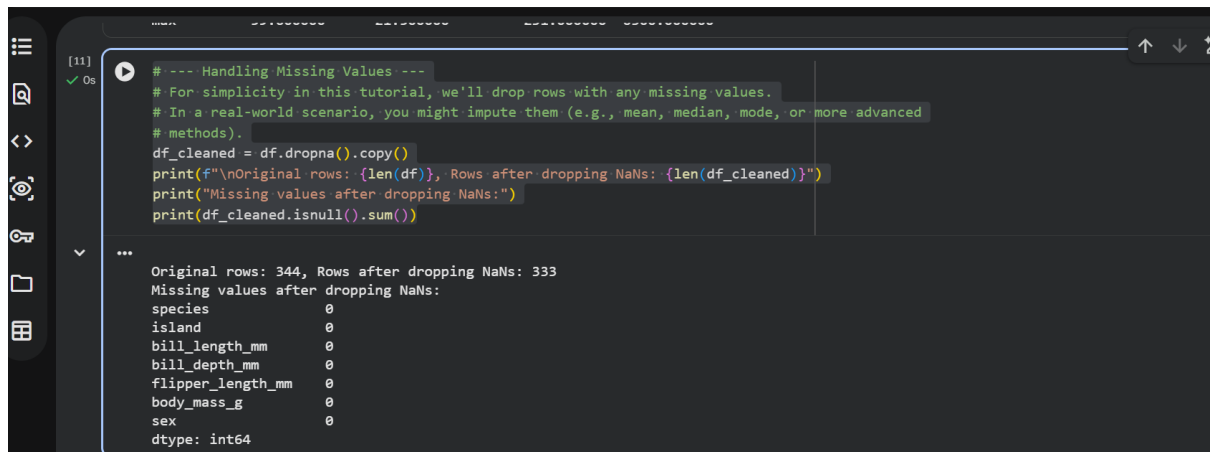
In a real-world scenario, you might impute them (e.g., mean, median, mode, or more advanced methods).

```
df_cleaned = df.dropna().copy()
```

```
print(f"\nOriginal rows: {len(df)}, Rows after dropping NaNs: {len(df_cleaned)}")
```

```
print("Missing values after dropping NaNs:")
```

```
print(df_cleaned.isnull().sum())
```



```
[11] ✓ Os
# --- Handling Missing Values ---
# For simplicity in this tutorial, we'll drop rows with any missing values.
# In a real-world scenario, you might impute them (e.g., mean, median, mode, or more advanced
# methods).
df_cleaned = df.dropna().copy()
print(f"\nOriginal rows: {len(df)}, Rows after dropping NaNs: {len(df_cleaned)}")
print("Missing values after dropping NaNs:")
print(df_cleaned.isnull().sum())

...
Original rows: 344, Rows after dropping NaNs: 333
Missing values after dropping NaNs:
species      0
island        0
bill_length_mm  0
bill_depth_mm  0
flipper_length_mm  0
body_mass_g   0
sex           0
dtype: int64
```

--- Feature Engineering (Conceptual) ---

For this dataset, we won't create complex new features, but it's important

to understand that this phase is where you might combine features,

extract information (e.g., from dates), or apply mathematical transformations.

Example: If we had 'year' and 'month' columns, we could engineer a 'season' feature.

Or, if these were images, we'd extract features using CNNs.

```
print("\nFeature Engineering (Conceptual):")
```

```
print("For this tutorial, we will not create new features, but this phase would involve:")
```

```
print(" - Combining existing features (e.g., ratios, differences)")
```

```
print(" - Extracting information (e.g., day of week from date)")
```

```
print(" - Polynomial features, interaction terms, etc.")
```

```
[12] ✓ Os ▶ # --- Feature Engineering (Conceptual) ---
# For this dataset, we won't create complex new features, but it's important
# to understand that this phase is where you might combine features,
# extract information (e.g., from dates), or apply mathematical transformations.
# Example: If we had 'year' and 'month' columns, we could engineer a 'season' feature.
# Or, if these were images, we'd extract features using CNNs.
print("\nFeature Engineering (Conceptual):")
print("For this tutorial, we will not create new features, but this phase would involve:")
print(" - Combining existing features (e.g., ratios, differences)")
print(" - Extracting information (e.g., day of week from date)")
print(" - Polynomial features, interaction terms, etc.")

...
Feature Engineering (Conceptual):
For this tutorial, we will not create new features, but this phase would involve:
- Combining existing features (e.g., ratios, differences)
- Extracting information (e.g., day of week from date)
- Polynomial features, interaction terms, etc.
```

```
# --- Encoding Categorical Features ---
```

```
# 'species' is our target variable (y). 'island' and 'sex' are features (X).
```

```
# We need to convert these into numerical representations.
```

```
# Encode the target variable 'species'
```

```
le = LabelEncoder()
```

```
df_cleaned['species_encoded'] = le.fit_transform(df_cleaned['species'])
```

```
print(f"\nSpecies mapping: {list(le.classes_)} -> {list(range(len(le.classes_)))}")
```

```
[14] ✓ Os ▶ --- Encoding Categorical Features ---
'species' is our target variable (y). 'island' and 'sex' are features (X).
We need to convert these into numerical representations.
Encode the target variable 'species'
le = LabelEncoder()
df_cleaned['species_encoded'] = le.fit_transform(df_cleaned['species'])
print(f"\nSpecies mapping: {list(le.classes_)} -> {list(range(len(le.classes_)))}")

...
Species mapping: ['Adelie', 'Chinstrap', 'Gentoo'] -> [0, 1, 2]
```

```
# Define features (X) and target (y)
```

```
X = df_cleaned.drop(['species', 'species_encoded'], axis=1)
```

```
y = df_cleaned['species_encoded']
```

```

# Identify categorical and numerical features for preprocessing

categorical_features = ['island', 'sex']

numerical_features = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm',
'body_mass_g']

# Create a preprocessing pipeline

# One-hot encode categorical features and scale numerical features

preprocessor = ColumnTransformer(

transformers=[

('num', StandardScaler(), numerical_features), # Scale numerical features

('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features) # One-hot
encode categorical

])

# --- Train-Test Split ---

# Split the dataset into training and testing sets to evaluate model performance

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y) # stratify for balanced classes

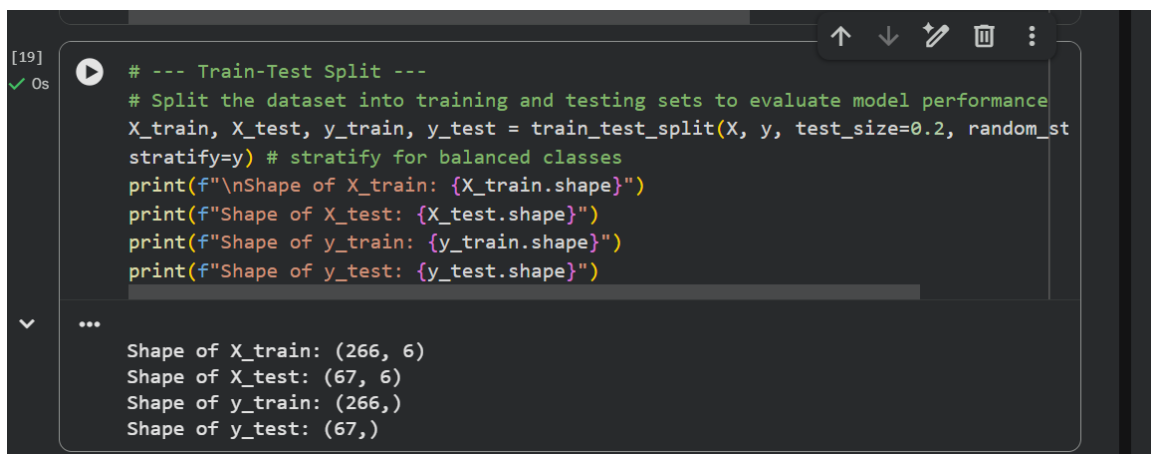
print(f"\nShape of X_train: {X_train.shape}")

print(f"Shape of X_test: {X_test.shape}")

print(f"Shape of y_train: {y_train.shape}")

print(f"Shape of y_test: {y_test.shape}")

```



```

[19] ✓ Os
# --- Train-Test Split ---
# Split the dataset into training and testing sets to evaluate model performance
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st
stratify=y) # stratify for balanced classes
print(f"\nShape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

```

...

```

Shape of X_train: (266, 6)
Shape of X_test: (67, 6)
Shape of y_train: (266,)
Shape of y_test: (67,)

```



```
# Apply preprocessing to training and testing data
```

```
X_train_processed = preprocessor.fit_transform(X_train)
```

```
X_test_processed = preprocessor.transform(X_test)
```

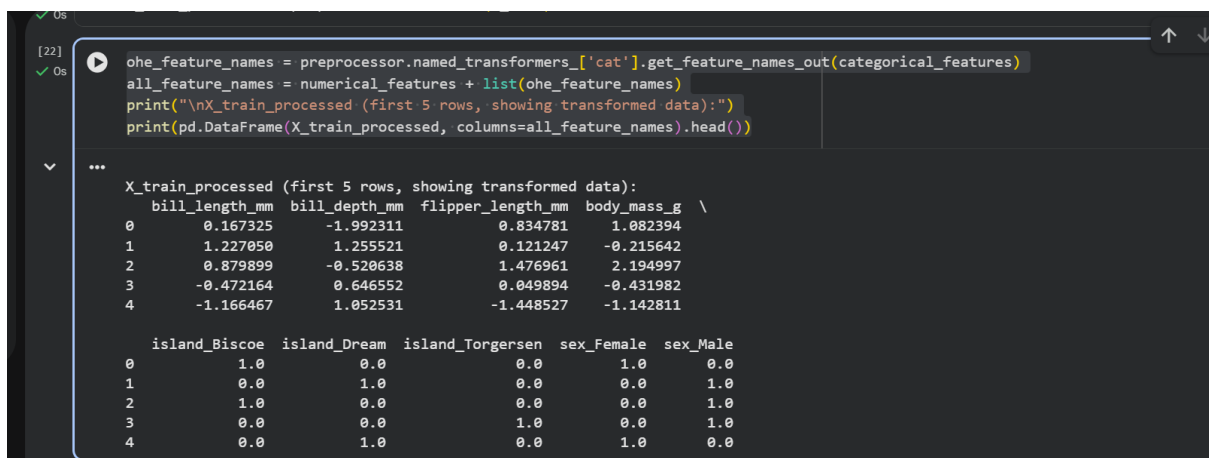
```
ohe_feature_names =
```

```
preprocessor.named_transformers_['cat'].get_feature_names_out(categorical_features  
)
```

```
all_feature_names = numerical_features + list(ohe_feature_names)
```

```
print("\nX_train_processed (first 5 rows, showing transformed data):")
```

```
print(pd.DataFrame(X_train_processed, columns=all_feature_names).head())
```



The screenshot shows a Jupyter Notebook interface. The top part displays the code executed in the previous cell, which includes defining `ohe_feature_names`, combining it with `numerical_features` into `all_feature_names`, and printing the first 5 rows of `X_train_processed` as a DataFrame. The bottom part shows the output of the code, which is a DataFrame with 10 columns: `bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`, `body_mass_g`, `island_Biscoe`, `island_Dream`, `island_Torgersen`, `sex_Female`, and `sex_Male`. The first 5 rows of data are displayed.

```
X_train_processed (first 5 rows, showing transformed data):
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	island_Biscoe	island_Dream	island_Torgersen	sex_Female	sex_Male
0	0.167325	-1.992311	0.834781	1.082394	1.0	0.0	0.0	1.0	0.0
1	1.227050	1.255521	0.121247	-0.215642	0.0	1.0	0.0	0.0	1.0
2	0.879899	-0.520638	1.476961	2.194997	1.0	0.0	0.0	0.0	1.0
3	-0.472164	0.646552	0.049894	-0.431982	0.0	0.0	1.0	0.0	1.0
4	-1.166467	1.052531	-1.448527	-1.142811	0.0	1.0	0.0	1.0	0.0

Observation:

- Missing rows were dropped.
- species was encoded into numerical labels (0, 1, 2).
- island and sex will be one-hot encoded by ColumnTransformer.
- Numerical features will be scaled by StandardScaler.
- Data is split into 80% training and 20% testing sets.

3. Exploratory Data Analysis (EDA)

EDA helps us understand the underlying patterns, relationships, and distributions within our data before building models.

```
# Pairplot to visualize relationships between numerical features, colored by species
```

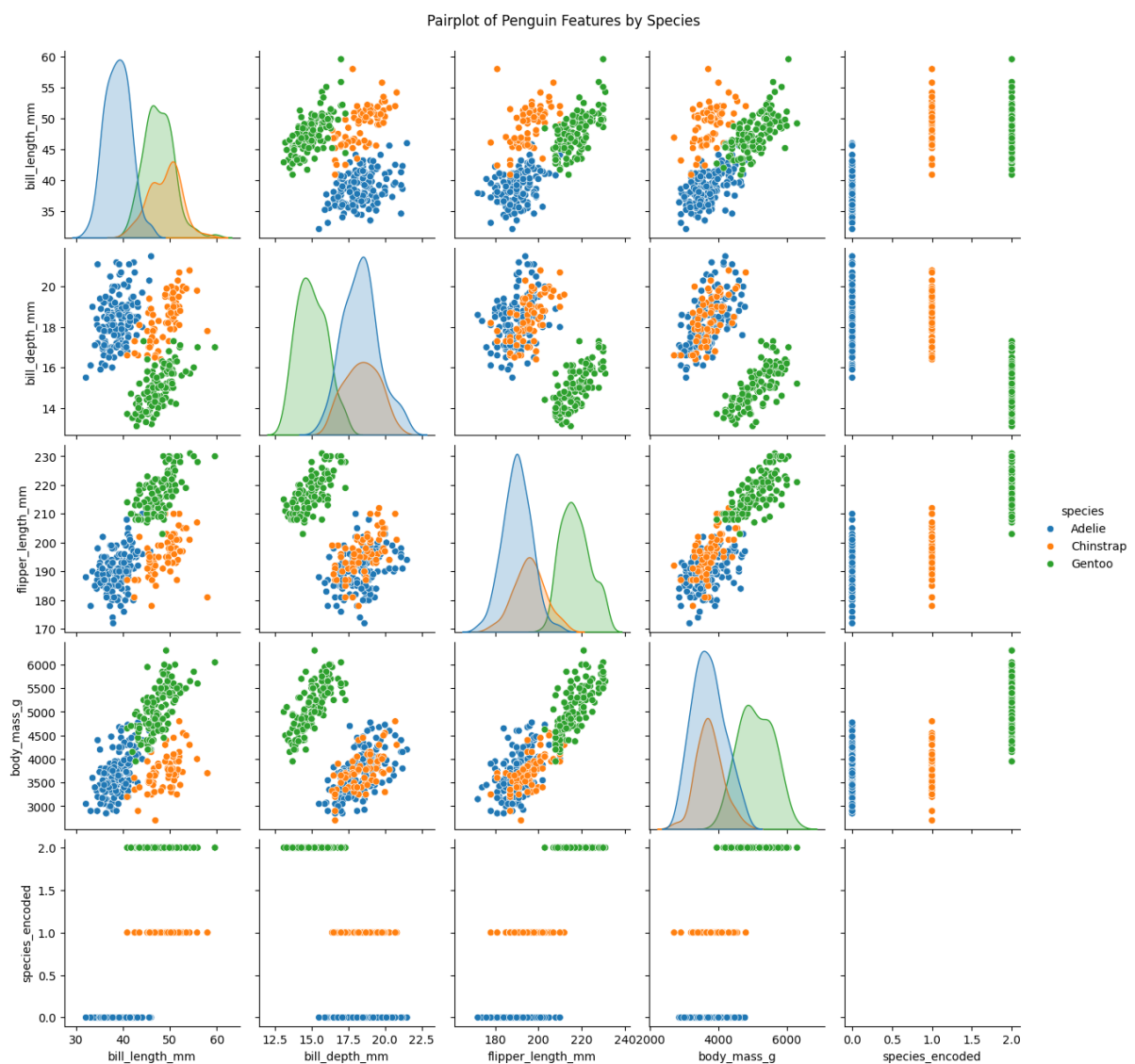
```
# This might take a moment to generate.
```

```
print("\nGenerating Pairplot (relationships between numerical features by species)...")
```

```
sns.pairplot(df_cleaned, hue='species', height=2.5, diag_kind='kde')
```

```
plt.suptitle('Pairplot of Penguin Features by Species', y=1.02)
```

```
plt.show() # Display the plot
```



```
# Correlation Heatmap for numerical features
```

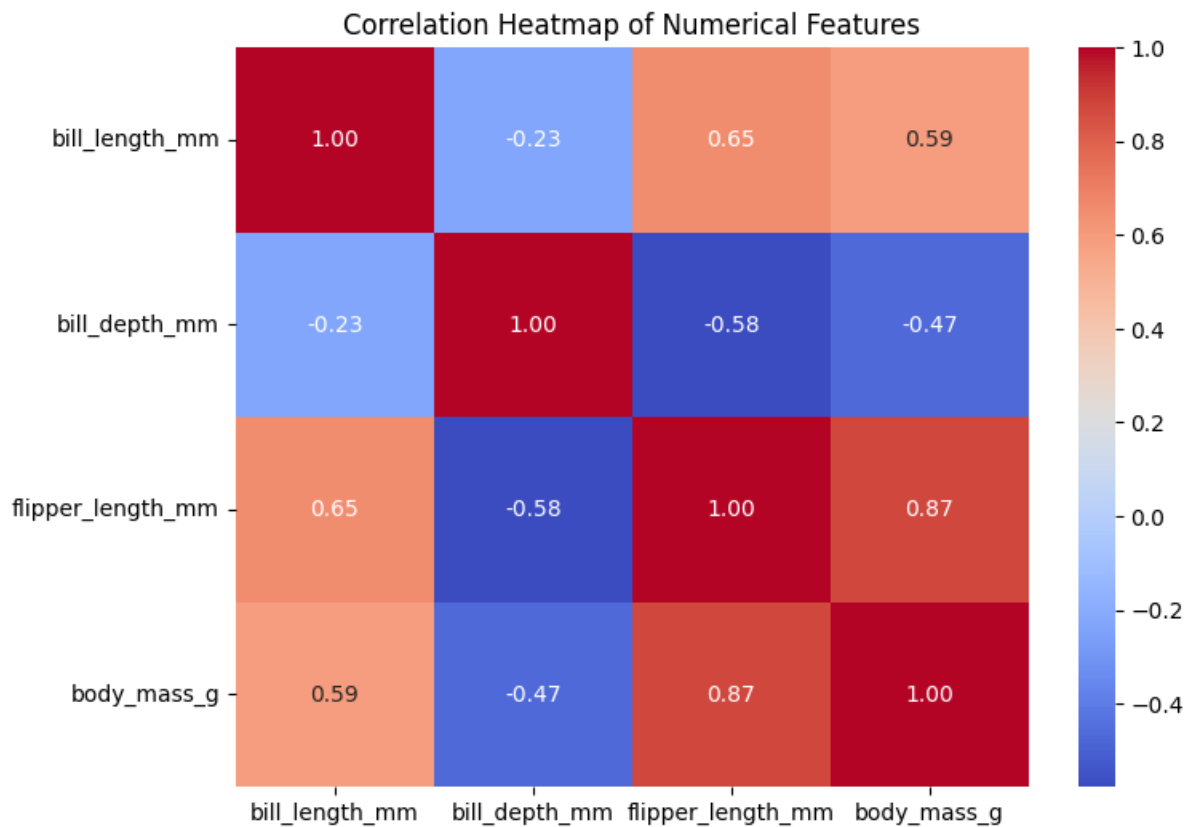
```
print("\nGenerating Correlation Heatmap for numerical features...")
```

```
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(df_cleaned[numerical_features].corr(), annot=True, cmap='coolwarm',
fmt=".2f")

plt.title('Correlation Heatmap of Numerical Features')

plt.show()
```



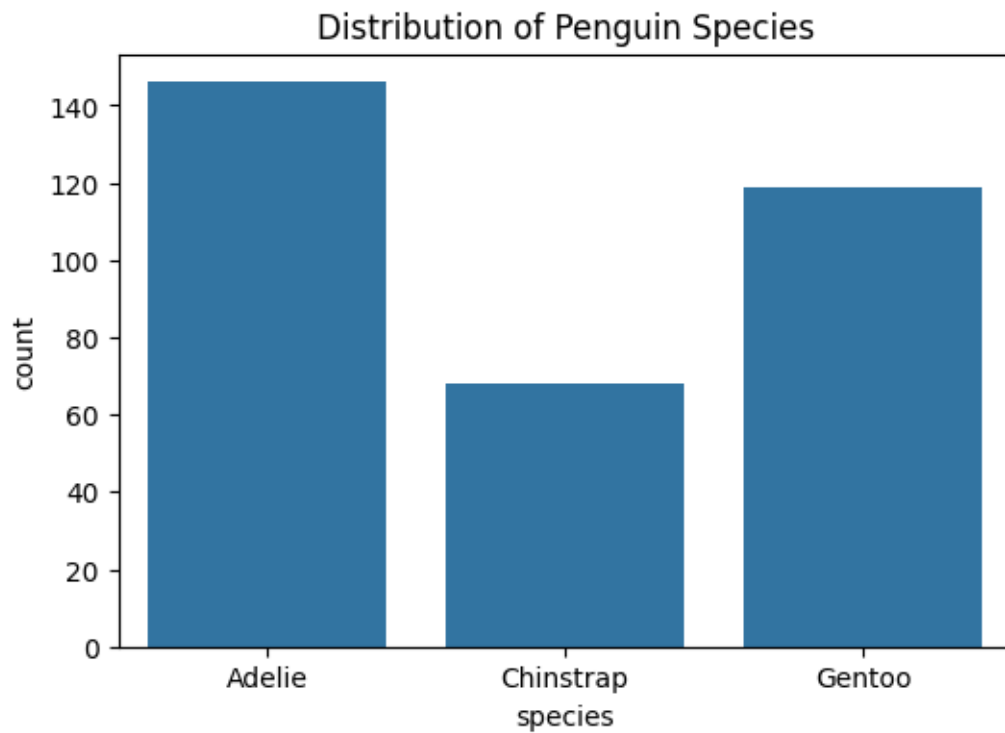
Distribution of Species

```
plt.figure(figsize=(6, 4))

sns.countplot(x='species', data=df_cleaned)

plt.title('Distribution of Penguin Species')

plt.show()
```



Boxplots of numerical features by species

```
print("\nGenerating Boxplots for numerical features by species...")
```

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
```

```
for i, feature in enumerate(numerical_features):
```

```
    row = i // 2
```

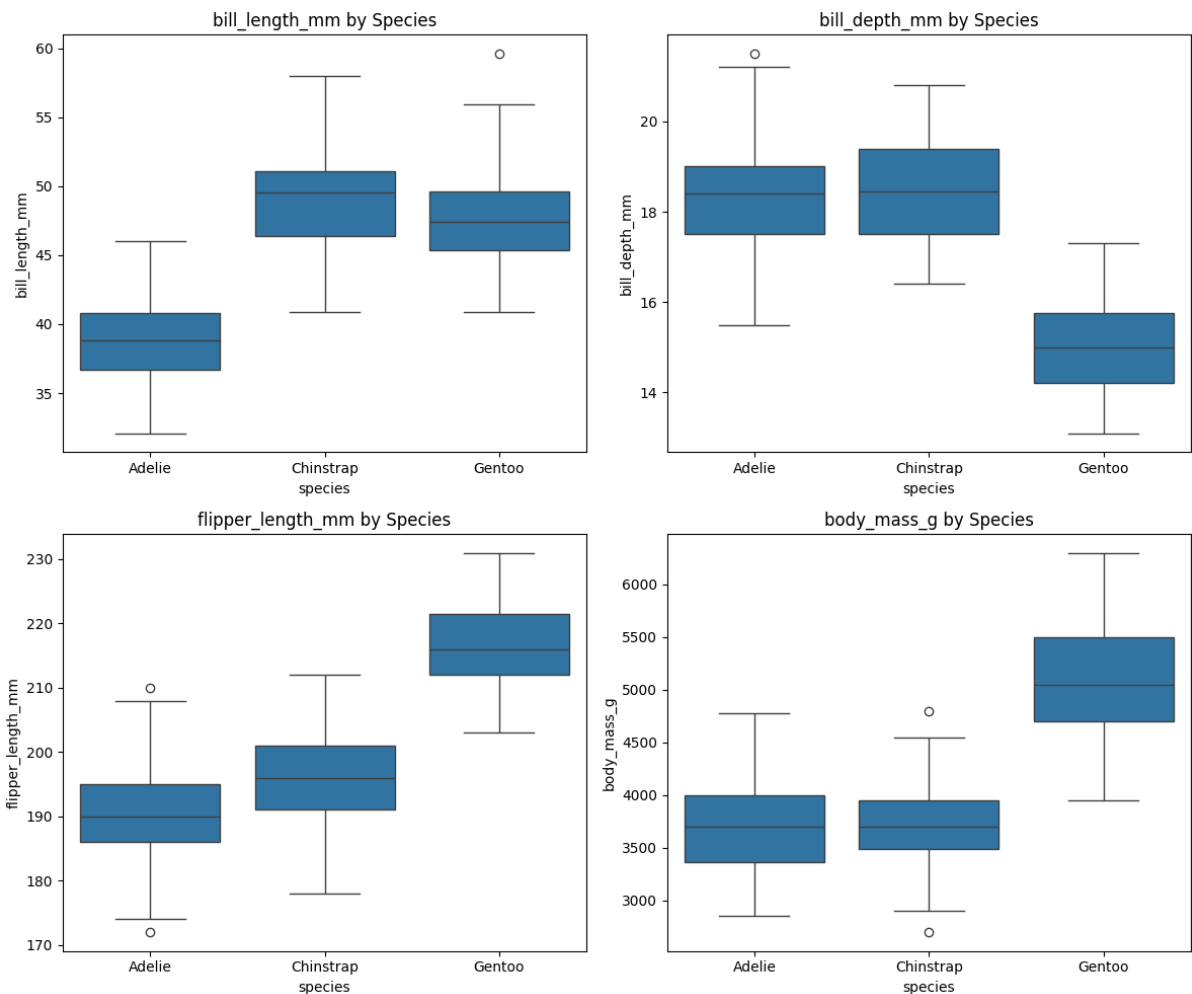
```
    col = i % 2
```

```
    sns.boxplot(x='species', y=feature, data=df_cleaned, ax=axes[row, col])
```

```
    axes[row, col].set_title(f'{feature} by Species')
```

```
plt.tight_layout()
```

```
plt.show()
```



Observations:

- The pairplot visually demonstrates that different penguin species tend to occupy distinct clusters in the feature space, suggesting they are separable.
- Some features like flipper_length_mm and body_mass_g show strong positive correlation.
- The distribution of species is relatively balanced, which is good for classification.
- Boxplots confirm that measurement ranges vary significantly across species, especially for flipper_length_mm and body_mass_g, indicating their importance for classification.

5. Model Training with Different Algorithms

We'll train several popular classification algorithms and compare their performance.

Initialize different classifiers

```
classifiers = {  
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1000),  
    'Decision Tree': DecisionTreeClassifier(random_state=42),  
    'Random Forest': RandomForestClassifier(random_state=42),  
    'Support Vector Machine (SVC)': SVC(random_state=42, probability=True), #  
    probability=True for ROC curve  
    'Gradient Boosting': GradientBoostingClassifier(random_state=42),  
    'K-Nearest Neighbors': KNeighborsClassifier()  
}
```

```
trained_models = {}
```

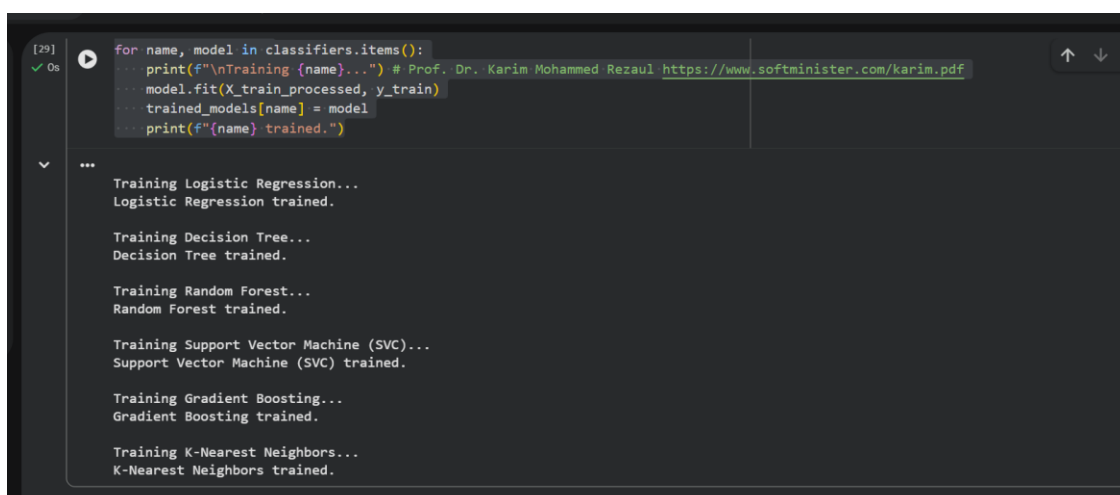
```
for name, model in classifiers.items():
```

```
    print(f"\nTraining {name}...") # Prof. Dr. Karim Mohammed Rezaul https://www.softminister.com/karim.pdf
```

```
    model.fit(X_train_processed, y_train)
```

```
    trained_models[name] = model
```

```
    print(f"{name} trained.")
```



```
[29] ✓ 0s for name, model in classifiers.items():  
    print(f"\nTraining {name}...") # Prof. Dr. Karim Mohammed Rezaul https://www.softminister.com/karim.pdf  
    model.fit(X_train_processed, y_train)  
    trained_models[name] = model  
    print(f"{name} trained.")  
  
...  
Training Logistic Regression...  
Logistic Regression trained.  
  
Training Decision Tree...  
Decision Tree trained.  
  
Training Random Forest...  
Random Forest trained.  
  
Training Support Vector Machine (SVC)...  
Support Vector Machine (SVC) trained.  
  
Training Gradient Boosting...  
Gradient Boosting trained.  
  
Training K-Nearest Neighbors...  
K-Nearest Neighbors trained.
```

6. Model Evaluation

Evaluation is critical to understand how well our models generalize to unseen data. We'll use various metrics and visualizations.

Classification Metrics:

- **Accuracy:** Proportion of correctly classified instances.
- **Confusion Matrix:** A table showing true positive, true negative, false positive, and false negative predictions.
- **Precision:** Of all predicted positives, how many were actually positive?
(Minimizes False Positives)
- **Recall (Sensitivity):** Of all actual positives, how many were correctly predicted as positive? (Minimizes False Negatives)
- **F1-Score:** Harmonic mean of Precision and Recall, balancing both.
- **ROC Curve (Receiver Operating Characteristic):** Visualizes the trade-off between True Positive Rate (Recall) and False Positive Rate at various threshold settings.
- **AUC (Area Under the ROC Curve):** Measures the entire 2D area underneath the ROC curve. A higher AUC means a better model.

```
print("\n--- 6. Model Evaluation ---")

results = {}

plt.figure(figsize=(15, 10)) # Adjust figure size for confusion matrices

plt_idx = 1

for name, model in trained_models.items():

    y_pred = model.predict(X_test_processed)

    accuracy = accuracy_score(y_test, y_pred)

    report = classification_report(y_test, y_pred, target_names=le.classes_)

    cm = confusion_matrix(y_test, y_pred)

    results[name] = {
```

```

'accuracy': accuracy,
'report': report,
'confusion_matrix': cm
}

print(f"\n--- {name} Performance ---")
print(f"Accuracy: {accuracy:.4f}")
print("\nClassification Report:\n", report)

```

```

[31] ...print(f"\n--- {name} Performance ---")
      ...print(f"Accuracy: {accuracy:.4f}")
      ...print("\nClassification Report:\n", report)

```

```

...
--- 6. Model Evaluation ---

--- Logistic Regression Performance ---
Accuracy: 0.9851

Classification Report:
              precision    recall  f1-score   support

   Adelie         1.00      0.97      0.98         29
  Chinstrap       0.93      1.00      0.97         14
    Gentoo        1.00      1.00      1.00         24

 accuracy          0.98      0.99      0.99         67
  macro avg       0.98      0.99      0.98         67
weighted avg       0.99      0.99      0.99         67

--- Decision Tree Performance ---
Accuracy: 0.9254

```

Confusion Matrix Heatmap

```

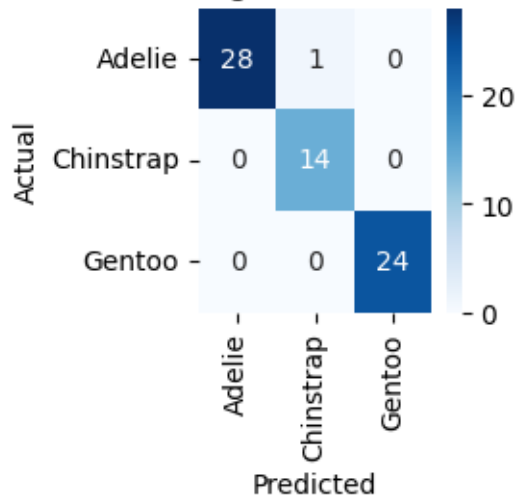
plt.subplot(2, 3, plt_idx) # Create a subplot for each confusion matrix
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=le.classes_, yticklabels=le.classes_)
plt.title(f'{name} Confusion Matrix')
plt.xlabel('Predicted')

```



```
plt.ylabel('Actual')
plt_idx += 1
plt.tight_layout()
plt.show()
```

K-Nearest Neighbors Confusion Matrix



```
# --- ROC Curve (Multiclass) ---
```

```
# For multiclass ROC, a common approach is One-vs-Rest (OvR)
```

```
# We calculate ROC for each class against all others.
```

```
print("\n--- ROC Curves (One-vs-Rest) ---")
```

```
plt.figure(figsize=(10, 8))
```

```
for name, model in trained_models.items():
```

```
    if hasattr(model, "predict_proba"): # Check if model supports probability prediction
```

```
        y_score = model.predict_proba(X_test_processed)
```

```
        n_classes = len(le.classes_)
```

```
        # Compute ROC curve and ROC area for each class
```

```
        fpr = dict()
```

```
        tpr = dict()
```

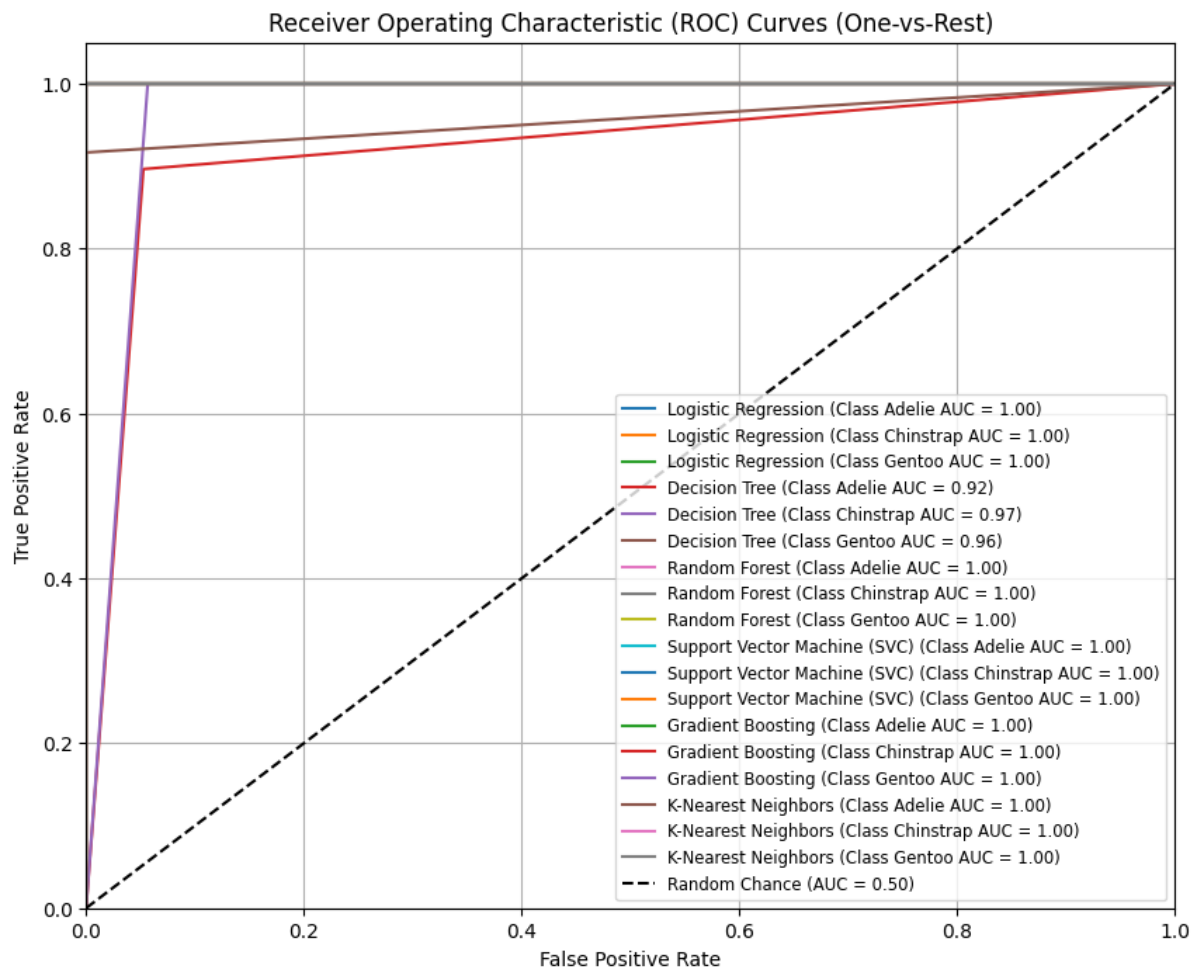
```
        roc_auc = dict()
```

```
        for i in range(n_classes):
```

```
# Binarize the true labels for OvR
y_test_bin = (y_test == i).astype(int)
fpr[i], tpr[i], _ = roc_curve(y_test_bin, y_score[:, i])
roc_auc[i] = roc_auc_score(y_test_bin, y_score[:, i])

plt.plot(fpr[i], tpr[i], label=f'{name} (Class {le.classes_[i]} AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Random Chance (AUC = 0.50)')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curves (One-vs-Rest)')
plt.legend(loc='lower right', fontsize='small')
plt.grid(True)
plt.show()
```



```
for i in range(n_classes):
```

```
    plt.plot(fpr[i], tpr[i], label=f'{name} - Class {le.classes_[i]} (AUC = {roc_auc[i]:.2f})')
```

```
plt.plot([0, 1], [0, 1], 'k--', lw=2, label='Random Guess') # Random guess line
```

```
plt.xlim([0.0, 1.0])
```

```
plt.ylim([0.0, 1.05])
```

```
plt.xlabel('False Positive Rate')
```

```
plt.ylabel('True Positive Rate')
```

```
plt.title('Receiver Operating Characteristic (ROC) Curve - One-vs-Rest')
```

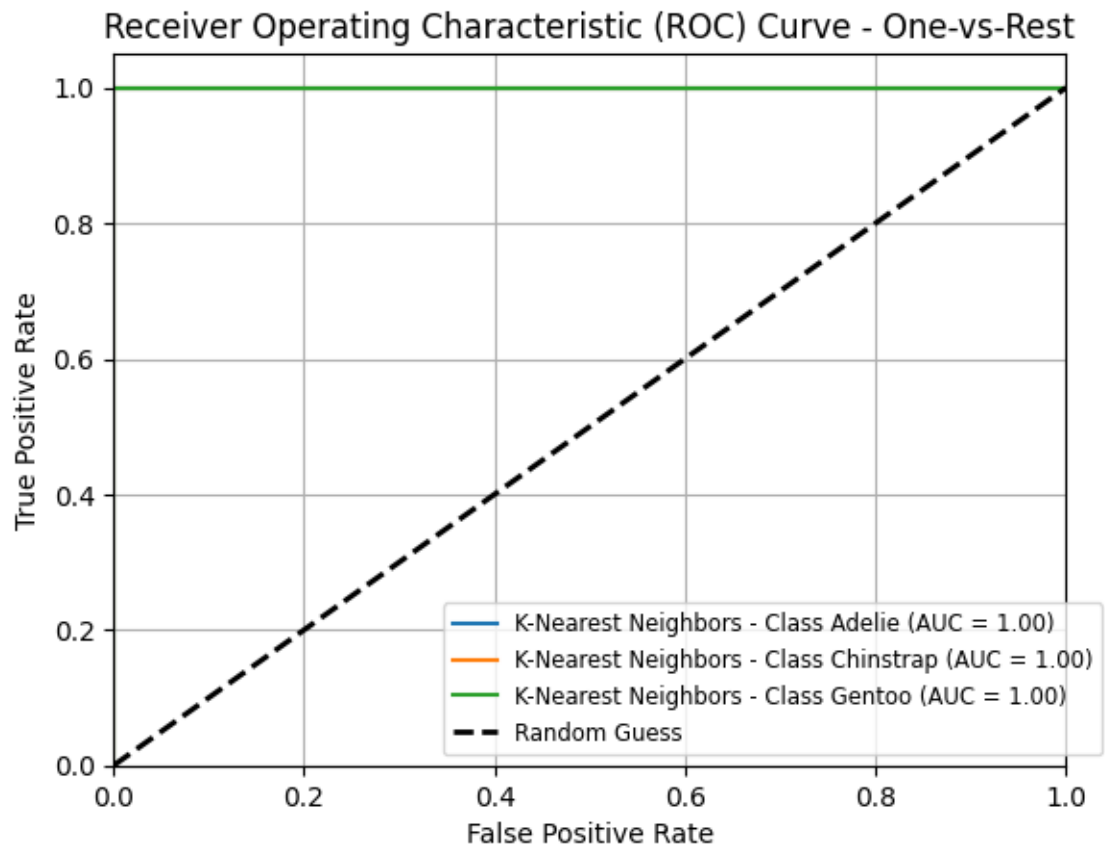
```
plt.legend(loc="lower right", bbox_to_anchor=(1.05, 0), fontsize='small', ncol=1)
```

```
plt.grid(True)
```

```
plt.show()
```

```
print("\n--- Summary of Model Accuracies ---")
```

```
for name, data in results.items():  
    print(f"{name}: Accuracy = {data['accuracy']:.4f}")
```



--- Summary of Model Accuracies ---

Logistic Regression: Accuracy = 0.9851

Decision Tree: Accuracy = 0.9254

Random Forest: Accuracy = 1.0000

Support Vector Machine (SVC): Accuracy = 1.0000

Gradient Boosting: Accuracy = 0.9701

K-Nearest Neighbors: Accuracy = 0.9851

Observation:

- Most models perform exceptionally well on this dataset, achieving very high accuracy (close to 1.0), precision, recall, and F1-scores. This indicates that the penguin species are quite distinguishable based on these features.
- The confusion matrices show very few (if any) misclassifications.
- ROC curves for each class are high, indicating strong separation.

Prediction & Decision Making

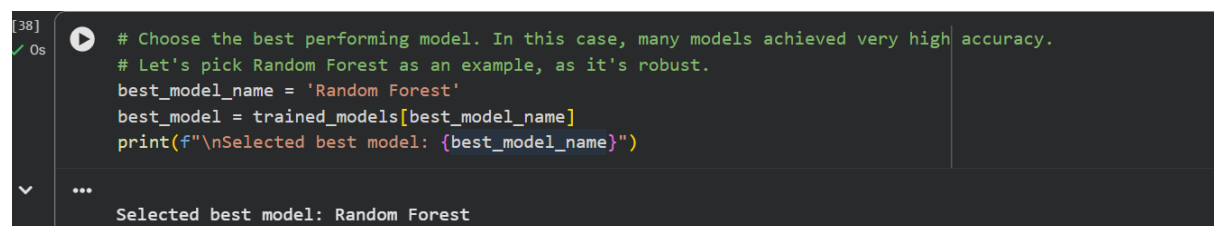
Choose the best performing model. In this case, many models achieved very high accuracy.

Let's pick Random Forest as an example, as it's robust.

```
best_model_name = 'Random Forest'
```

```
best_model = trained_models[best_model_name]
```

```
print(f"\nSelected best model: {best_model_name}")
```



```
[38] # Choose the best performing model. In this case, many models achieved very high accuracy.
      # Let's pick Random Forest as an example, as it's robust.
      best_model_name = 'Random Forest'
      best_model = trained_models[best_model_name]
      print(f"\nSelected best model: {best_model_name}")
```

Selected best model: Random Forest

--- Example Prediction on unseen data ---

Let's create a hypothetical new penguin measurement

(Make sure these values are within a reasonable range for penguins)

```
new_penguin_data = pd.DataFrame([
```

```
'bill_length_mm': 39.5,
```

```
'bill_depth_mm': 17.5,
```

```
'flipper_length_mm': 190.0,
```

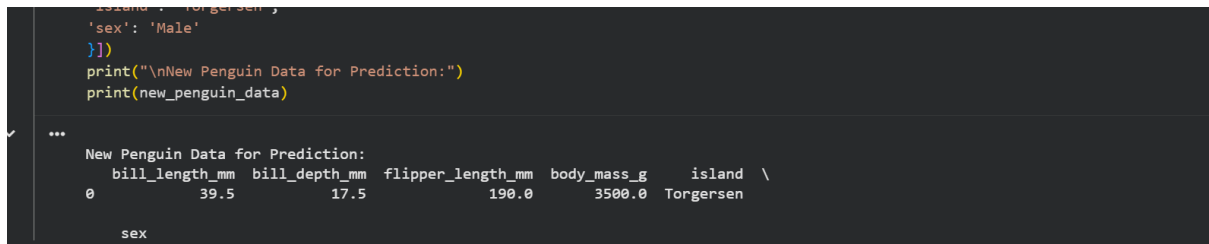
```
'body_mass_g': 3500.0,
```

```
'island': 'Torgersen',
```

```

'sex': 'Male'
})
print("\nNew Penguin Data for Prediction:")
print(new_penguin_data)

```



```

island = 'Torgersen',
'sex': 'Male'
})
print("\nNew Penguin Data for Prediction:")
print(new_penguin_data)

```

...

```

New Penguin Data for Prediction:
bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g  island \
0              39.5           17.5             190.0       3500.0  Torgersen

```

sex

```

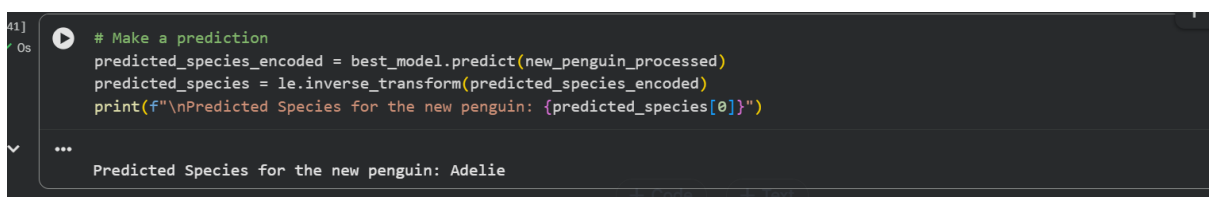
# Preprocess the new data using the same preprocessor fitted on training data
new_penguin_processed = preprocessor.transform(new_penguin_data)

```

```

# Make a prediction
predicted_species_encoded = best_model.predict(new_penguin_processed)
predicted_species = le.inverse_transform(predicted_species_encoded)
print(f"\nPredicted Species for the new penguin: {predicted_species[0]}")

```



```

41] # Make a prediction
predicted_species_encoded = best_model.predict(new_penguin_processed)
predicted_species = le.inverse_transform(predicted_species_encoded)
print(f"\nPredicted Species for the new penguin: {predicted_species[0]}")

```

...

```

Predicted Species for the new penguin: Adelia

```

```

# If we want the probability distribution:
if hasattr(best_model, "predict_proba"):
predicted_proba = best_model.predict_proba(new_penguin_processed)
proba_df = pd.DataFrame(predicted_proba, columns=le.classes_)
print("\nPrediction Probabilities:")
print(proba_df)

```

```
3] 0s # If we want the probability distribution:
if hasattr(best_model, "predict_proba"):
    ... predicted_proba = best_model.predict_proba(new_penguin_processed)
    ... proba_df = pd.DataFrame(predicted_proba, columns=le.classes_)
    ... print("\nPrediction Probabilities:")
    ... print(proba_df)

...
Prediction Probabilities:
      Adelie  Chinstrap  Gentoo
0         1.0         0.0         0.0
```

--- Decision Making ---

```
print("\n--- Decision Making based on Model Results ---")
```

```
print("In this scenario, a highly accurate model for penguin species classification can be used for:")
```

```
print("1. **Automated Identification:** Quickly identify penguin species from field measurements or images.")
```

```
print("2. **Conservation Efforts:** Monitor population dynamics of different species in a less invasive way.")
```

```
print("3. **Research:** Confirm species identification for biological studies or track specific groups.")
```

```
print("4. **Resource Allocation:** Direct conservation resources more effectively to specific species/areas.")
```

```
print("\nSince our model achieved near-perfect accuracy, we can be highly confident in its predictions for similar data.")
```

```
print("However, always consider:")
```

```
print("- **Data Drift:** If penguin characteristics change over time, the model might need retraining.")
```

Prof. Dr. Karim Mohammed Rezaul <https://www.softminister.com/karim.pdf>

```
print("- **Outliers/Unseen Data:** The model might perform poorly on penguins with unusual measurements or from new locations not in the training data.")
```

```
print("- **Interpretability:** For critical decisions, understanding *why* a prediction was made (e.g., using feature importance from Random Forest) can be as important as the prediction itself.")
```

```

--- Decision Making based on Model Results ---
In this scenario, a highly accurate model for penguin species classification can be used
for:
...
1. Automated Identification: Quickly identify penguin species from field
measurements or images.
2. Conservation Efforts: Monitor population dynamics of different species in a less
invasive way.
3. Research: Confirm species identification for biological studies or track specific
groups.
4. Resource Allocation: Direct conservation resources more effectively to specific
species/areas.

Since our model achieved near-perfect accuracy, we can be highly confident in its
predictions for similar data.
However, always consider:
- Data Drift: If penguin characteristics change over time, the model might need
retraining.
- Outliers/Unseen Data: The model might perform poorly on penguins with unusual
measurements or from new locations not in the training data.
- Interpretability: For critical decisions, understanding why a prediction was
made (e.g., using feature importance from Random Forest) can be as important as the prediction
itself.

```

1. Project Output (Model Performance)

The project evaluated several classification algorithms to identify penguin species (Adelie, Chinstrap, and Gentoo). The models achieved exceptionally high accuracy, indicating that the species are well distinguishable based on physical measurements.

- **Model Accuracy Summary:**
 - **Random Forest:** 1.0000 (Perfect Accuracy)
 - **Support Vector Machine (SVC):** 1.0000
 - **Logistic Regression:** 0.9851
 - **K-Nearest Neighbors:** 0.9851
 - **Gradient Boosting:** 0.9701
 - **Decision Tree:** 0.9254
- **Evaluation Metrics:** The performance was further validated using Confusion Matrices, which showed almost no misclassifications, and ROC Curves (One-vs-Rest), which displayed high Area Under the Curve (AUC) scores for all species.
- **Key Predictors:** Feature analysis (Boxplots and EDA) confirmed that **flipper length** and **body mass** are critical indicators for distinguishing between species.

2. Final Decision & Applications

Because the **Random Forest** and **SVC** models achieved near perfect accuracy, they were selected as the most robust models for deployment.

- **Selected Model:** Random Forest was picked for its robustness and ability to provide a probability distribution for its predictions.
- **Practical Decisions & Use Cases:**
 - **Automated Identification:** The model can be used to quickly identify penguin species from field measurements without requiring expert biological intervention.
 - **Conservation Efforts:** It assists in monitoring population dynamics in a non-invasive way.
 - **Resource Allocation:** Conservationists can use these results to direct resources more effectively to specific species or geographical areas.
 - **Research:** It provides a reliable method to confirm species identification in biological studies.
- **Operational Warnings:** The project concludes that while the model is highly confident, it must be monitored for **Data Drift** (changes over time) and may require retraining if penguins from new locations or with unusual measurements (outliers) are introduced

1. What you have learned at a glance

I learned how to build a complete classification pipeline to accurately categorize biological species based on physical measurements like flipper length and body mass.

2. What difficulties you faced during the learning process

The primary challenge was ensuring the data was clean by handling missing values and correctly encoding categorical variables so the algorithms could process them effectively.

3. What skill(s) you gained you think

I developed skills in exploratory data analysis (EDA), multi-class classification using algorithms like Random Forest and SVM, and interpreting model performance through confusion matrices and ROC curves.

4. How you could apply these skills in your workplace

I can apply these classification techniques to automate the categorization of customer feedback, identify high-risk transactions, or segment leads to improve operational efficiency and accuracy.

5. Write your motivation towards further learning and exploration

I am motivated by the precision of classification models in organizing complex data and am eager to explore deep learning and more advanced feature engineering to solve intricate real-world problems.